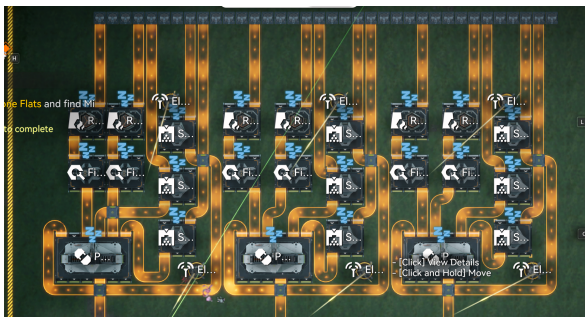
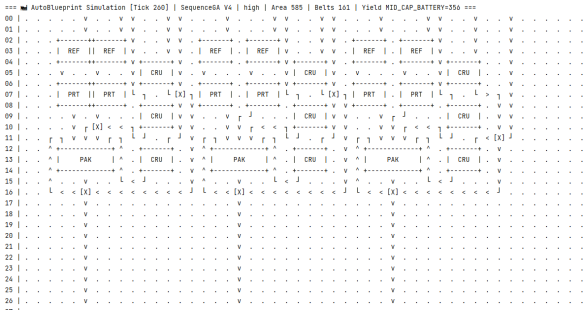
AutoBlueprint: Autonomous Agent Design for Compact Factory Blueprint Generation

Zecheng Tang 20513308
COMP3004 Designing Intelligent Agents

May 2026



Actual blueprint



SequenceGA output

Contents

Abstract

AutoBlueprint is an autonomous agent system for generating compact and physically valid factory blueprints. The environment models machines, recipes, ports, transport belts, and grid constraints. The project compares a rule-based baseline with search and optimisation agents using best area, best belt length, routing success, and successful production of the required output amount.

1 Introduction

Factory automation games and production simulators pose a useful model problem for intelligent agents: a designer must transform a target product and rate into a valid arrangement of machines and transport components. This requires reasoning about recipes, throughput, space, ports, and routing.

AutoBlueprint addresses this problem by treating blueprint generation as an autonomous agent task. Given a target product, the agent perceives a symbolic production environment, decides which machines and transport components are required, places them on a grid, and attempts to connect the resulting material flows.

1.1 Research question

How do different autonomous blueprint-generation agents compare in producing compact, valid, and routable factory layouts under the same production requirements?

1.2 Contributions

- A grid-based factory environment with explicit machines, materials, ports, recipes, and transport components.
- Several autonomous agents for blueprint generation, including a rule-based baseline and optimisation-based variants.
- An experimental comparison using measurable layout and routing metrics.

2 Background and Related Work

2.1 Agents, Layout, and Search

An autonomous agent perceives an environment and selects actions to achieve a task [11]. In AutoBlueprint, percepts are recipes, machine properties, ports, and grid occupancy; actions are instantiation, placement, orientation, and routing. Since one placement can block later belts, the task requires deliberation.

Factory layout generation is related to facility layout, where machines are arranged under spatial constraints while reducing material-handling cost [4]. AutoBlueprint is smaller and game-oriented, but keeps the same trade-off between compactness, flow efficiency, and access space. Sequence-pair representations from rectangle packing and VLSI placement [10] are relevant because the buildings are rectangular modules with relative ordering constraints.

2.2 Routing and Optimisation

Routing is graph search over grid cells, with buildings and existing transport as obstacles. A* combines path cost with a distance heuristic [7]. For placement, simulated annealing explores large spaces by sometimes accepting worse moves early [8]. AutoBlueprint combines recipe reasoning, placement search, and routing validation.

3 System Overview

AutoBlueprint is a controlled production-layout environment. Given a target product and rate, an agent produces a grid blueprint with machines, transport, and directed material flows. Recipes define the production graph; the grid tests whether it can be placed and routed under size, port, collision, and compactness constraints.

3.1 Environment Model

The `GridMap` class represents a fixed-size two-dimensional grid. Cells may be empty, occupied by a building, or occupied by transport. The environment checks boundaries, collisions, rotation-dependent size, bridge endpoints, and active ports. Tick updates move items through buildings and transport, so blueprints are tested as working layouts rather than static drawings.

3.2 Entity and Recipe Representation

The environment is built from materials, buildings, and transport components. Table 1 lists the attributes used for production and feasibility reasoning.

Table 1: Core attributes of the main entity types.

Entity type	Main attributes	Purpose in the system
Material	Material Type; Material State, either solid or liquid.	Defines produced or transported items and whether belts or pipes are legal.
Building	Component ID; Name; Size; System Type; Input Materials; Output Materials; Production Speed; Inventory Capacity; Power Fields; Direction; Input Side; Output Side; Active Input Ports; Active Output Ports; Direct Insertion Flag; Omni Port Flag.	Defines footprint, recipe behaviour, throughput, ports, and direct insertion.
Transport component	Component ID; Name; Position; Direction; Current Item; Next Tick Item; Maximum Capacity; System Type; Supported Material State; Optional Bridge End Position; Component Specific Routing Logic.	Moves material through straight belts, splitters, mergers, crossers, access parts, and bridges.

Materials distinguish solid and liquid items. Buildings store recipes and ports. Transport components form the logistics network, and `registry` returns fresh catalogue objects so inventories and buffers are not shared.

3.3 Two Logistics Systems

System A follows *Mindustry*, a factory-management and tower-defense game with conveyor logistics [14, 9]. It uses omni-directional ports and may allow direct insertion. System B follows *Arknights: Endfield*, whose industrial systems include automated chains and conveyor routing [13, 2]. Its buildings have directional ports and require access parts, belts, pipes, splitters, mergers, or crossers.

All evaluated agents use System B because its explicit transport-space requirement better exposes planning ability.

3.4 Task Definition

The agent must infer the production chain, instantiate machines, place and orient them, and connect material flows. A valid blueprint stays inside the map, avoids overlap, respects material state and port rules, and can produce the target. The objective is validity, compactness, and routability.

3.5 Agent-Environment Interaction

The interaction follows a perceive-plan-act-evaluate cycle: read the catalogue and target, build a production graph, place and route entities, then evaluate area, belt length, routing, and production.

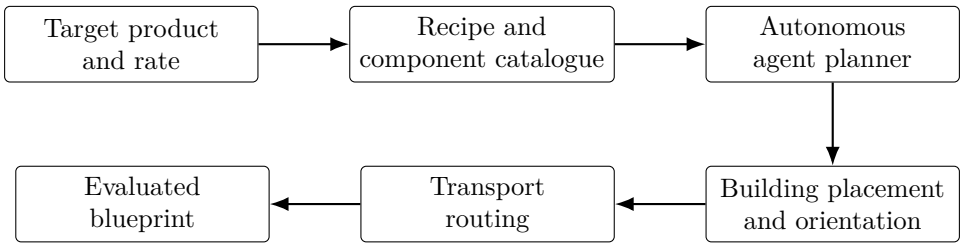


Figure 1: High-level AutoBlueprint system flow. The agent transforms a symbolic production requirement into a physical grid blueprint and receives feedback through validity and quality metrics.

4 Agent Design

All agents transform a symbolic target into a physical System B blueprint. The versions are iterative: each keeps useful earlier mechanisms while addressing area, routing, blocked ports, congestion, or instability.

4.1 Shared Pipeline and Production Graph

Most agents build a production graph, search for positions and directions, commit the layout, and route each logical edge. Later agents feed routing information back into layout search because compact placement is useful only when transport can also be placed legally.

The production graph is generated backwards from the target. Raw inputs terminate expansion; other materials require a recipe, producer instances, and upstream demands. Early agents use round robin matching, while later versions add capacity quotas or min cost flow.

Algorithm 1 Backward production graph construction

Initialise demand queue with the target output requirements
while demand queue is not empty **do**
    Pop material m and required amount a
    if m is an available raw input **then**
        Mark m as an external input
    else
        Find recipe r and producer building for m
        Instantiate $\lceil a/\mathrm{rate}(r, m) \rceil$ producer buildings
        Add scaled input demands of r to the queue
    end if
end while
Match producer instances to consumer instances using the family specific rule
return directed production graph

4.2 Agent Families and Version Evolution

The families progress from deterministic placement to routing-aware optimisation and richer layout representations.

Table 2: Agent versions and algorithmic ideas.

Agent version	Algorithm / logic
`GenericBaselineAgent`	Rule-based block layout with A* routing.
`SABaselineAgent` / SA V1	Coordinate-based simulated annealing.
`SAAgentV2`	Routing-aware simulated annealing with failure feedback.
`FdpSaAgent` V1	Force-directed placement followed by SA refinement.
`FdpSaAgentV2`	Multi-start routing-aware FDP+SA.
SequenceSA V1 (`SequencePairSaAgentV1`)	Sequence-pair module floorplanning.
SequenceSA V2 (`SequencePairSaAgentV2`)	Production-cell B*-tree packing.
SequenceSA V3 (`SequencePairSaAgentV3`)	Port-aware rotation and compaction search.
SequenceGA V4 (`SequencePairGaAgentV4`)	Genetic fan in cell search with detailed beam routing.
`FlowCpAgentV1`	Min-cost-flow assignment with CP-style cell packing.

4.3 Baseline Agent

The baseline computes required buildings, partitions them around final-product roots, and places producers before consumers. Its A* router uses floating start and goal ports. It works on small chains but handles congestion, directional ports, and tight corridors poorly.

4.4 Simulated Annealing Agents

The simulated annealing agents perturb a layout and accept moves according to cost and temperature. Later states include both position and direction.

Algorithm 2 Simulated annealing placement loop

Initialise a valid or partially valid layout
Set initial temperature T
while termination condition not met **do**
    Propose a neighbouring layout by moving or rotating a building
    Compute cost difference Δ
    if $\Delta < 0$ or $random(0, 1) < \exp(-\Delta/T)$ **then**
        Accept the proposed layout
    else
        Keep the previous layout
    end if
    Reduce temperature T
end while
return best layout found

SA V1 replaces deterministic placement with coordinate search, penalising invalid footprints, area, and long links. SA V2 adds direction, side-specific ports, input/output edge bias, and failure feedback.

The final SA version evaluates candidate layouts with:

$$C_{\mathrm{SA2}} = 1.2A + 4D_p + 8000O + 10000(B + P) + 40H_{io} + C_{\mathrm{fb}}, \tag{1}$$

where A is area, D_p is nearest port distance, O overlap count, B out-of-bounds count, P blocked ports, H_{io} input/output edge distance, and C_{fb} failed-route feedback.

Algorithm 3 Routing-aware simulated annealing with failure feedback

```
Build a capacity-aware production DAG
for each layout retry do
    Clear the grid and routing state
    Run simulated annealing using current routing penalties
    Place the best candidate layout
    Attempt detailed routing with rip-up and reroute
    if all routes are connected then
        return successful blueprint
    else
        Increase penalties for failed routing tasks
    end if
end for
return best partially routed blueprint
```

4.5 FDP and Routing-Aware SA Agents

FDP+SA adds a force-directed stage before annealing, pulling connected buildings together and pushing unrelated buildings apart [5]. FDP V1 evaluates overlap, area, port distance, alignment, turns, and crossings. FDP V2 adds layered seeds, multi-start annealing, trial routing, endpoint repair, local expansion, compaction, and crosser upgrades.

The final FDP version uses an annealing layout cost:

$$C_{\mathrm{FDP2}} = w_A(t)A + w_E(t)E + 6|W - H| + 10D_p + P_{\mathrm{legal}} + 12H_{io}, \tag{2}$$

where E is empty box area, W, H are box size, t is progress, and P_{legal} covers boundary, overlap, and blocked ports. After trial routing:

$$S_{\mathrm{FDP2}} = (F,\ A + 0.15R,\ R,\ C_{\mathrm{FDP2}})\,, \tag{3}$$

where F is failed routes and R is routed transport cells.

4.6 SequenceSA and SequenceGA Variants

SequenceSA uses simulated annealing over sequence-pair or B*-tree-style representations [3]. SequenceGA keeps production cells but searches them with genetic selection, crossover, and mutation [6].

SequenceSA V1 groups buildings into modules, optimises sequence-pair permutations, and applies route-preserving compaction. V2 changes modules into production cells with B* tree packing and negotiated congestion routing. V3 adds port-aware rotation and compaction.

The final SequenceSA version reuses the production cell layout feedback cost:

$$C_{\text{SeqSA}} = 20A + 1.5E + 7D_m + 900D_{\text{reverse}} + 18\sum_c \max(0, u_c - 1)^2 + 100000B, \tag{4}$$

where D_m is Manhattan flow distance, D_{reverse} penalises reverse flow, and u_c is estimated corridor usage. SequenceSA V3 compares route scores as:

$$S_{\text{SeqSA}} = (F,\ A,\ R,\ G,\ C_{\text{SeqSA}})\,, \tag{5}$$

with congestion G used after validity, area, and route length.

SequenceGA V4 uses genetic search over cell order, variants, and rotations. It also adds topology-aware fan in cells: if an upstream building supplies exactly one downstream building, the pair is placed in the same block. This prevents independent suppliers being pushed to blueprint edges. Cheap layout cost filters candidates before routing; beam routing, rip up and reroute, caching, rotation search, and compaction improve scalability.

The final SequenceGA version ranks routed candidates by:

$$S_{\text{SeqGA}} = (F,\ A + 0.12R + 0.08G,\ R)\,. \tag{6}$$

Before scoring, the fan in rule changes the layout space. For edge $u \to v$, if u only outputs to v, then u is placed inside v's local block:

$$\text{group}(u) = \text{group}(v) \quad \text{if} \quad \text{outdegree}(u) = 1 \text{ and } (u, v) \in E. \tag{7}$$

The detailed router scores paths as:

$$C_{\text{path}}(p) = |p| + 1.5T(p) + \sum_{x \in p} q_x + 0.25 \sum_{x \in p} h_x, \tag{8}$$

where $T(p)$ is the number of turns, q_x is learned congestion, and h_x is corridor penalty.

Algorithm 4 Genetic production-cell layout search

```
Identify one to one supplier consumer pairs in the production graph
Build fan in production cells around each downstream sink
Generate seed layouts from production cell variants
Initialise a population of genomes
for each generation do
    for each genome do
        Decode genome into a building layout
        Compute cheap layout feedback cost
    end for
    Select top candidates for trial routing
    Score routed candidates by S_SeqGA
    Preserve elite genomes
    Create new genomes using crossover and mutation
end for
Refine best layout with budgeted rotation search and local compaction
return best routed layout
```

4.7 Flow and CP Agent

FlowCpAgentV1 replaces round robin matching with min cost flow assignment before placement [1]. It also uses bounded CP-style refinement inside production cells.

Algorithm 5 Min-cost-flow provider-consumer assignment

for each material with providers and consumers **do**
    Create a source node and a sink node
    Add provider nodes with supply capacities
    Add consumer nodes with demand capacities
    Add provider-consumer edges with assignment costs
    Run min-cost flow until all demand units are assigned
    Convert used flow edges into production graph edges
end for
return balanced material-flow graph

For one material, provider i and consumer j are connected by flow f_{ij} with assignment cost:

$$C_{\text{flow}} = \min \sum_{i,j} f_{ij} \left(1000 \left| \frac{i}{\max(1, n_p - 1)} - \frac{j}{\max(1, n_c - 1)} \right| + b_{ij} \right), \tag{9}$$

where b_{ij} is a small type bias. CP cell refinement minimises:

$$C_{\text{cp}} = 1000A + 20W + 20H + 2 \sum \text{wire} + 80 \sum \text{down}, \tag{10}$$

under non-overlap constraints.

4.8 Detailed Routing

After placement, routing selects compatible ports and runs A*. Later routers add turn, crossing, corridor, and congestion penalties, plus beam alternatives and rip up rounds. Trial routes score layouts, repair endpoints, guide compaction, and validate blueprints.

5 Implementation and Evaluation Method

The system is implemented in Python. Domain data, grid validation, planning, and tests are separated across `entities/`, `environment/grid_map.py`, `agents/`, and `test/`.

5.1 Implementation Challenges

The main challenge was aligning symbolic production with physical grid constraints. A logical connection may fail if a port is blocked, a footprint overlaps, or no legal belt or pipe path exists. Early area-only costs produced compact but unroutable layouts, so later agents added port, failure, congestion, turn, crossing, and repair terms.

5.2 Experimental Question

The evaluation asks whether optimisation agents produce more valid, compact, and routable blueprints than the deterministic baseline under the same System B requirements.

5.3 Conditions

All agents use the same catalogues, grid size, and targets. The baseline is run once per target; each randomised family uses five seeds. Low, medium, and high form the main benchmark, with SequenceGA also tested on extreme.

Table 3: Benchmark production targets.

Difficulty	Target product and amount	Available raw inputs
Low	2 × BLUE_IRON_BOTTLE	BLUE_IRON
Medium	1 × MID_CAP_BATTERY	BLUE_IRON, ORIGINIUM
High	3 × MID_CAP_BATTERY	BLUE_IRON, ORIGINIUM
Extreme	1 × HIGH_CAP_BATTERY	BLUE_IRON, ORIGINIUM, SANDLEAF_SEED

The diagrams show production flows: arrows are materials and amounts; boxes are building counts.

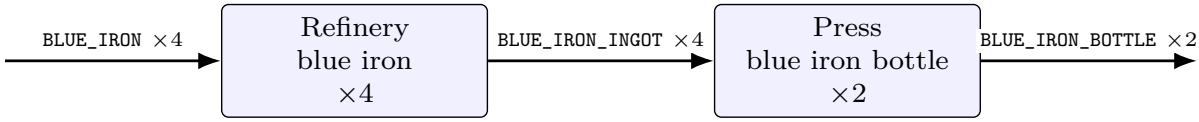


Figure 2: Low difficulty production chain: 2 BLUE_IRON_BOTTLE.

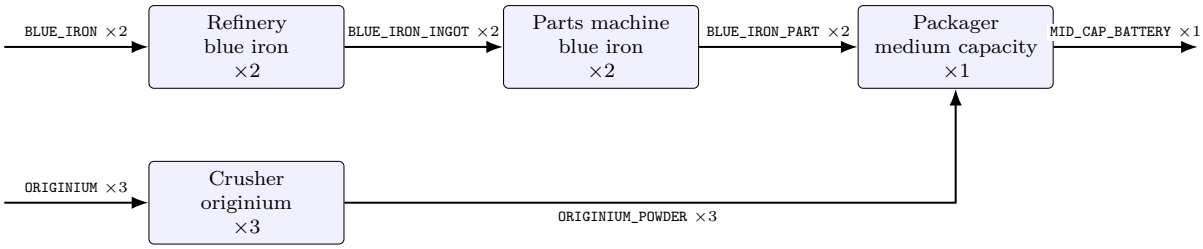


Figure 3: Medium difficulty production chain: 1 MID_CAP_BATTERY.

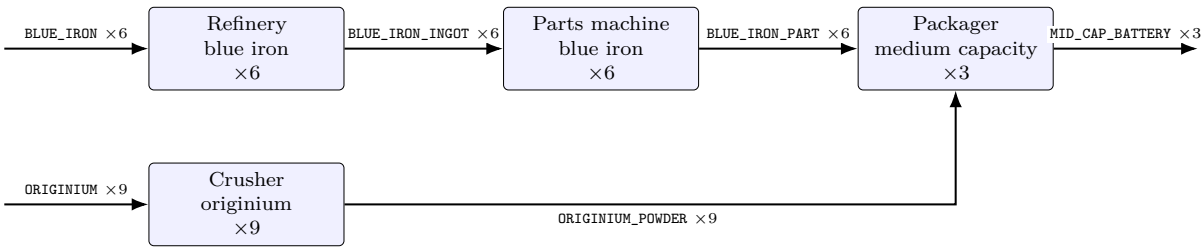


Figure 4: High difficulty production chain: 3 MID_CAP_BATTERY.

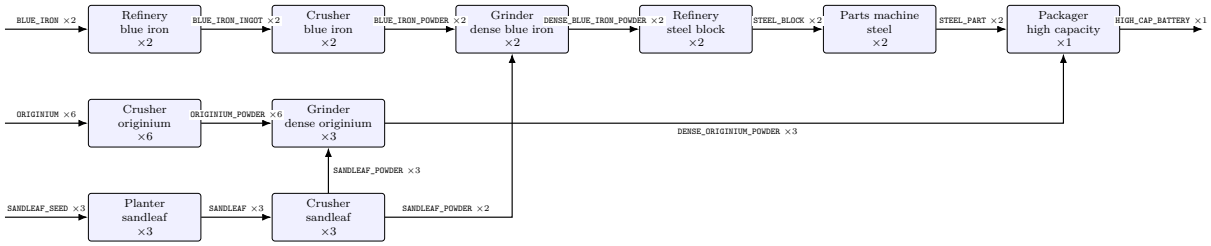


Figure 5: Extreme difficulty production chain: 1 `HIGH_CAP_BATTERY`.

5.4 Metrics

Table 4: Evaluation metrics and calculation methods.

Metric	Calculation method	Meaning
Best Area	Minimum bounding-box area among valid or best available runs.	Spatial footprint.
Best Belt Length	Minimum placed transport components among valid or best available runs.	Logistics length.
Routing Success / 5	Seeds where all material-flow edges connect.	Routability.
Production Success / 5	Seeds where the target amount is produced.	Production validity.

5.5 Procedure

Each blueprint is evaluated with the same checks. Invalid or partially routed layouts are kept because failure rate matters. The report records best area, best belt length, routing success, production success, and qualitative examples.

6 Results

Table 5: Main benchmark results across three difficulty levels.

Difficulty	Family	Best Area ↓	Best Belt Length ↓	Routing Success / 5 ↑	Production Success / 5 ↑
Low	Baseline	312	32	100%	100%
Low	SA	440	132	100%	100%
Low	FDP+SA	140	27	100%	100%
Low	SequenceSA	170	48	100%	100%
Low	SequenceGA	112	22	100%	100%
Low	Flow/CP	120	27	100%	100%
Medium	Baseline	621	104	100%	100%
Medium	SA	285	49	100%	100%
Medium	FDP+SA	272	66	100%	100%
Medium	SequenceSA	165	46	100%	100%
Medium	SequenceGA	165	45	100%	100%
Medium	Flow/CP	208	66	100%	100%
High	Baseline	FAIL	FAIL	0%	0%
High	SA	FAIL	FAIL	0%	0%
High	FDP+SA	FAIL	FAIL	0%	0%
High	SequenceSA	816	241	100%	100%
High	SequenceGA	585	161	100%	100%
High	Flow/CP	1269	310	100%	100%

Table 6: SequenceGA extreme test.

Test	Best Area ↓	Best Belt Length ↓	Routing Success / 5 ↑	Production Success / 5 ↑
Extreme	3069	539	100%	100%

6.1 Best Blueprint Examples

The examples show best generated blueprints. SequenceGA also includes an in-game screenshot. A compact legend precedes the ASCII renders.

Table 7: Legend for simulated blueprint examples.

Symbol	Meaning
`REF`	Refinery.
`CRU`	Crusher.
`PRT`	Parts machine.
`PRS`	Press.
`PAK`	Packager.
`GRD`	Grinder.
`PLT` / `EXT`	Planter / extractor.
`.`	Empty cell.
`> < ^ v`	Belt direction.
Corner belt cell	Belt turn.
`[X]`	Crosser.
`[S]`	Splitter/router.
`[M]`	Merger.
`X01`, `S01`, or direction plus number	Component currently carrying one item.

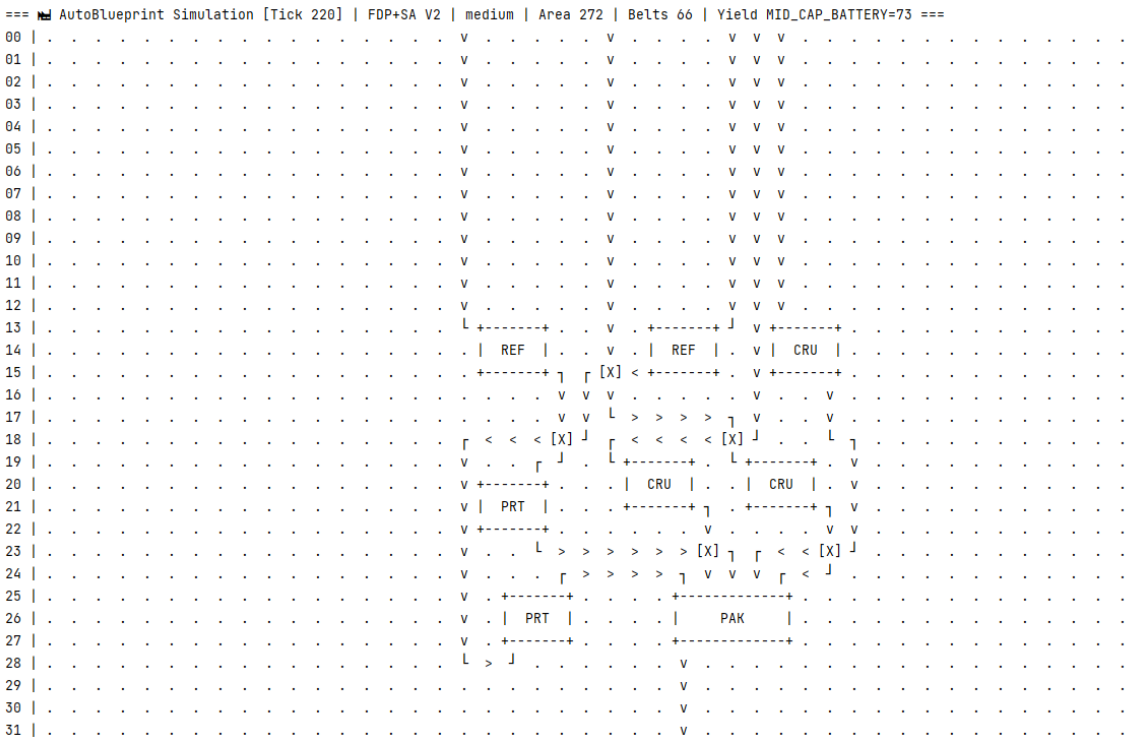


Figure 6: Example Result: FDP_SA simulated blueprint on medium difficulty.

Figure 7: Eample Result: SequeceSA simulated blueprint on high difficulty.

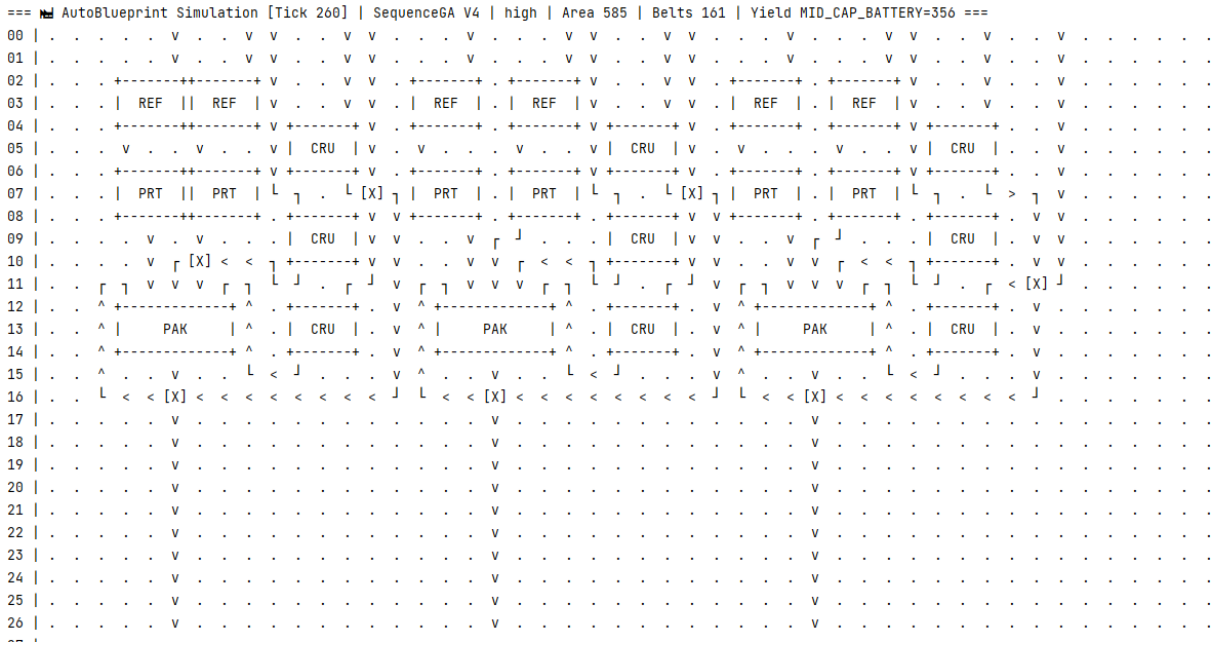


Figure 8: Example Result: SequenceGA simulated blueprint on high difficulty.

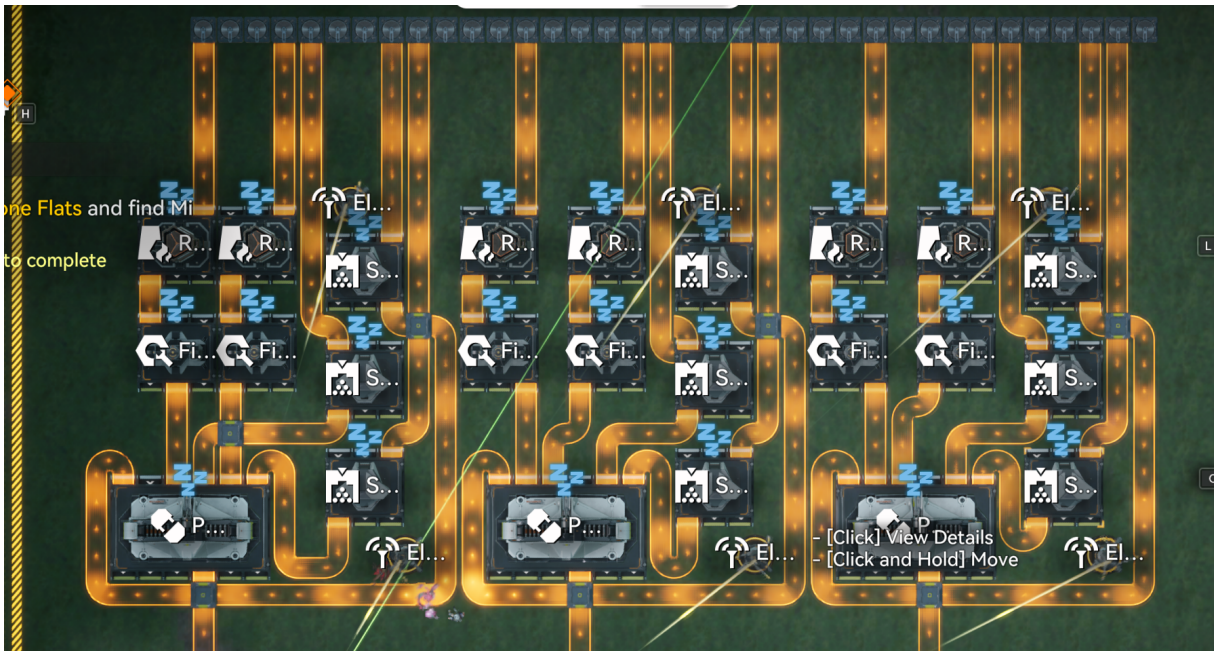


Figure 9: Real Blueprint in Game: SequenceGA simulated blueprint on high difficulty.

```
=== AutoBlueprint Simulation [Tick 132] | SequenceGA V4 | high | Area 585 | Belts 161 | Yield MID_CAP_BATTERY=179 ===
```

Figure 10: Working Status: SequenceGA simulating blueprint on high difficulty.

7 Discussion

The results show that AutoBlueprint is not only a placement problem, but a joint reasoning problem over recipes, ports, routing, and simulation. A compact rectangle is useful only when the belts can still be routed and the simulated factory can produce the requested amount. This

explains why the best agents are not the ones with the simplest area objective, but the ones that combine production graph structure with routing feedback.

7.1 Answer to the Research Question

The research question asks whether autonomous agents can generate compact, routable, and productive System B blueprints. The answer is positive, but only when symbolic production reasoning is coupled with routing-aware spatial search. The deterministic baseline works on easier chains but fails on high difficulty, while SA and FDP+SA improve flexibility without fully solving dense multi-input routing.

Sequence-based agents perform better because they search over production cells rather than isolated coordinates. SequenceSA reaches a valid high-difficulty result, and SequenceGA gives the strongest result in the table: best area 585 and best belt length 161, with full routing and production success. Its fan in cell rule keeps one to one suppliers near their consumers, reducing both edge placement and belt length. Thus, the best performance comes from combining optimisation with blueprint-specific structure.

7.2 Strengths

The strongest engineering feature is the extensible environment model. The `grid_map`, `registry`, material catalogue, recipe catalogue, building model, and transport components are separated cleanly, so changed products or new buildings, materials, and recipes require local catalogue updates rather than agent rewrites.

The environment also contains transport logic and tick-based simulation for belts, ports, crossers, generated inputs and outputs, inventories, and production ticks. A generated blueprint can therefore be tested as a working material-flow system, making evaluation closer to the real game environment than a purely geometric check.

The agent design is also broad: rule-based placement, simulated annealing, force-directed placement, sequence-pair search, B* tree packing, genetic search, negotiated routing, and min cost flow assignment are compared in one framework. This shows a progression from simple heuristics to hybrid agents, especially in SequenceGA, where evolutionary search is built on meaningful production cells.

7.3 Limitations

Result quality depends strongly on the target product. A layout rule that works well for one chain may be poor for another with different fan in, fan out, port pressure, or raw input demand. This makes it difficult to build one evaluation system that is fair for all products; area, belt length, routing success, and production success are useful but do not always agree.

Computational cost is another limitation. More detailed agents spend more time on routing trials, repair, rotation search, and simulation. As building count grows, the combinations of positions, orientations, ports, and routes grow rapidly, so high-difficulty tasks are hard to solve with a single pure algorithm.

Finally, the environment is still an abstraction. It captures the main System B constraints used here, but not every game mechanic or export format.

7.4 Finding

A key finding is that SA and GA parameter tuning alone gives limited improvement once the search is stable. Larger gains came from adding domain knowledge, such as SequenceGA's fan in cell rule, which encodes the observation that a one to one supplier should usually be near its consumer. Effective agents therefore need both general search and blueprint-specific construction rules.

8 Conclusion and Future Work

8.1 Conclusion

AutoBlueprint demonstrates that factory blueprint generation can be formulated as an autonomous intelligent agent problem. It builds an extensible System B environment with catalogues, building models, transport components, and tick-based simulation, so agent outputs can be checked as working production systems rather than only static layouts.

The experiments show that compact, routable, and productive blueprints can be generated when production planning, placement search, routing, and simulation are linked. SequenceGA gives the strongest high-difficulty result because it combines genetic search with production-cell structure and the fan in rule. Overall, the project shows that hybrid methods are most suitable: optimisation explores alternatives, while domain knowledge gives the search a useful structure.

8.2 Future Work

The main future direction follows from the finding that experience-based blueprint rules can matter more than parameter tuning. Good rules improve initial layouts and reduce computation by avoiding poor search regions. This suggests that reinforcement learning could perform well, because an agent could learn reusable placement and routing policies from repeated blueprint attempts [12]. It was not completed here because the required training time, memory, and computation exceeded the project limits.

Future work will add more blueprint-specific rules, including common subchain grouping, input and output corridor selection, fan in and fan out placement, and crosser usage. Another extension is to complete the game's building and product catalogue, allowing wider and more realistic benchmarks.

A more authoritative evaluation scheme is also needed. A full benchmark should consider more products, map shapes, game-specific constraints, and a clearer balance between area, belt length, production rate, routing complexity, and robustness. Finally, System A could be completed to compare the Mindustry-style logistics model with System B and study how transport rules change planning difficulty.

AI Statement

AI tools were used to assist with report structure planning and minor language refinement. All components were first written by author and were further checked after AI modified.

References

[1] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[2] Arknights Terra Wiki contributors. Arknights: Endfield. https://arknights.wiki.gg/wiki/Arknights:_Endfield, 2026. Accessed 13 May 2026.

[3] Yun-Chih Chang, Yao-Wen Chang, Guang-Ming Wu, and Shu-Wei Wu. B*-trees: A new representation for non-slicing floorplans. In *Proceedings of the 37th Design Automation Conference*, pages 458–463, 2000.

[4] Amine Drira, Henri Pierreval, and Sonia Hajri-Gabouj. Facility layout problems: A survey. *Annual Reviews in Control*, 31(2):255–267, 2007.

[5] Peter Eades. A heuristic for graph drawing. *Congressus Numerantium*, 42:149–160, 1984.

[6] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.

[7] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[8] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[9] Mindustry Wiki contributors. Mindustry wiki. https://mindustry.fandom.com/wiki/Mindustry_Wiki, 2026. Accessed 13 May 2026.

[10] Hiroshi Murata, Kunihiro Fujiyoshi, Shigetoshi Nakatake, and Yoji Kajitani. Rectangle-packing-based module placement. *Proceedings of the IEEE International Conference on Computer-Aided Design*, pages 472–479, 1995.

[11] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

[12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

[13] Wikipedia contributors. Arknights: Endfield. https://en.wikipedia.org/wiki/Arknights:_Endfield, 2026. Accessed 13 May 2026.

[14] Wikipedia contributors. Mindustry. https://en.wikipedia.org/wiki/Mindustry, 2026. Accessed 13 May 2026.